

Reefer Bird $\Rightarrow Q$ or $B' \Rightarrow \lambda f g x.g(f x)$ i.e. **reversed composition**

Mockingbird $\Rightarrow \omega \equiv \lambda x.xx$ i.e. applying a function to itself

Omega $\Rightarrow \Omega \equiv \omega \omega$ is a *non-terminating* combinator which has no *head normal form* (let alone *β -normal form*) i.e. its *unsolvable*

Warbler $\Rightarrow W \equiv \lambda f x.f x x$ i.e. **argument duplicator**;
e.g. $W (+) x \equiv x + x \equiv 2x$

Cardinal $\Rightarrow C \equiv \lambda f x y.f y x$ i.e. **flip**

Viero $\Rightarrow V \equiv \lambda x y z.z x y$ i.e. **pair constructor**

Starling $\Rightarrow S \equiv \lambda f g x.(f x)(g x)$ provides *context of x* to f, g } think: *reader monad*

Psi $\Rightarrow \Psi \equiv \lambda f g x y.f(g x)(g y)$ processes two arguments before combining them;
e.g. $(\Psi (+) \text{length})$ will add lengths of two lists

Recursion > Fixpoint Combinators

Fixed-Point of a function is where it returns the unchanged input
i.e. for any x , if $f x = x$ then x is a *fixed-point* e.g. $\text{double } 0 = 0$

Fixed-Point Combinator fix is such that:
if f has any *fixed-points*, then $\text{fix } f$ is one of those *fixed-points*, i.e. $f(\text{fix } f) =_{\beta} \text{fix } f$

These fixed point operators can be used to implement **recursion** in λ -calculus, e.g. **factorial** function fact }
 $\text{fact} \equiv \lambda n.\text{if } (\text{isZero } n) \text{ then } 1 \text{ else } n * (\text{fact } (n - 1))$

Now we can simply extract
 $\text{fact} \equiv [\lambda f n.\text{if } (\text{isZero } n) \text{ then } 1 \text{ else } n * (f (n - 1))] \text{ fact}$

The inner part we can call E }
i.e. $F \equiv \lambda f n.\text{if } (\text{isZero } n) \text{ then } 1 \text{ else } n * (f (n - 1))$

So now its $\text{fact} \equiv F \text{ fact}$

Now we use fix to re-write it as $\text{fact} \equiv \text{fix } E$ }
Since $\text{fix } F =_{\beta} F(\text{fix } F)$ and $\text{fix } F \equiv \text{fact}$ we get
 $\text{fact} =_{\beta} F \text{ fact}$ i.e. our *original* definition

Y-Combinator $\Rightarrow Y \equiv B \omega (C B \omega) \equiv \lambda f.(\lambda x.f(x x))(\lambda x.f(x x))$ }
 $Y f =_{\beta} f(Y f)$ i.e. both β -reduce to the same common term

However $f(Y f)$ may not in general β -reduce to $Y f$ }

Z-Combinator $\Rightarrow Z \equiv \lambda f.(\lambda x.f(\lambda y.x y))(\lambda x.f(\lambda y.x y))$ }
 $Z f x = f(Z f x)$ i.e. next argument is defined *explicitly*.

β -expansion of *Y-Combinator*, which will work for *strict languages* (*Y-Combinator* will stack overflow)

Turing (U) and Theta (Θ) combinators:
 $U \equiv \lambda x y.y(x x y)$ & $\Theta \equiv U U$

These mirror the definitions of *Mockingbird* and *Omega* combinators
 $\Theta f \rightarrow_{\beta} f(\Theta f)$ }
i.e. (unlike *Y-combinator*) it actually β -reduces